

as tags and fields and pushed to InfluxDB for additional analysis. Once the data has been processed and stored, the log file is emptied in anticipation of future log entries.

System features and functionalities

Overview of the application: The application is an event management platform where users can view several events happening nearby, select events of interest to participate in, and organise or host events. Users can also see the participant list for events they are hosting and for events in which they are participating. The application's backend is developed with Express.js and MongoDB, whereas the frontend is created with the React framework.

User authentication: Authentication is done through JWT, ensuring that only legitimate users can create events, join them, and access protected routes. Login and registration are processed through specific API routes that verify user credentials and provide tokens.

Real-time monitoring and analytics: The integration with InfluxDB allows the collection of performance metrics from the system. The following metrics are gathered:

- **CPU usage:** Monitors the percentage of CPU utilisation to detect high loads or resource contention.
- **Memory usage:** Tracks memory consumption, helping identify memory leaks or excessive resource usage.
- **Network usage:** Measures the network traffic to ensure the system can handle the data flow efficiently.
- **Disk I/O:** Tracks disk read/write operations, critical for performance in data-intensive applications.
- **System load:** Monitors the overall load on the system, providing insights into performance bottlenecks.

The collected metrics are periodically stored in InfluxDB and

visualised through Grafana dashboards. These dashboards enable real-time monitoring of CPU usage, memory consumption, and other critical system parameters.

API logging: The server logs every API request, including the endpoint, HTTP method, status code, and response time. This logging helps identify slow-performing endpoints and improve system performance over time. The logs are stored in a file (*api.log*), making it easy to track request and response data for debugging and optimisation.

Error handling and resilience: The system includes comprehensive error handling to catch and handle errors gracefully. If a request fails (e.g., due to a database issue or incorrect input), the server responds with appropriate error messages and status codes. This ensures a smooth user experience and helps with troubleshooting.

Random API tester: For simulating actual usage and testing the system's robustness, a random API tester was created. The tester carries out a few actions to make sure the backend can deal with various real-world situations:

- **User authentication:** The tester logs in with predefined credentials and obtains a JWT token for subsequent authenticated requests.
- **API invocation:** After authentication, the tester randomly selects an API endpoint (such as retrieving events, hosted events, or participation details) and initiates corresponding requests.
- **Variable request load:** The script generates a random number of API requests, ranging from 1 to 20, to simulate diverse traffic patterns and user interactions.
- **Authorisation handling:** All API requests incorporate the JWT token to ensure proper user authentication and session management.
- **Metrics integration:** The tester additionally triggers a data push to InfluxDB, enabling real-time updates of system performance

metrics.

The random API tester assists in simulating various user actions and loading conditions, and it gives significant information about the system's performance and reliability under different scenarios. It is a major tool used to detect potential bottlenecks or errors that can occur in real-world use.

Implementation details and results

Backend implementation: The backend uses Node.js and Express, whereby the system will respond to incoming API requests as well as create user sessions. Core implementations consist of:

- **JWT authentication:** Authenticated user limitation on the resources being accessed.
- **API routes:** Methods of handling authentication, event creation, and management.
- **Logging middleware:** Traps request information, response status, and time elapsed to handle each request.
- **InfluxDB integration:** Real-time data capture for system performance monitoring.

The live monitoring of the application involved instantaneous measures of various performance metrics like CPU core usage, memory usage, application logs, API response time, and response volume as indicated below. These were measured at fixed intervals during the application runtime, providing valuable insights into system performance and usage of resources with time.

CPU usage metrics analysis:

The system metrics were captured and written into InfluxDB to track the CPU performance. The mean values for the most important CPU indicators—*idle*, *iowait*, *system*, and *user*—were retrieved in the last hour, displayed in 10-second intervals, by means of a query. The outcome was represented as a pie chart in Grafana, as presented in Figure 1.